

# Improving Communication Between Pair Programmers Using Shared Gaze Awareness

Nikhil Anand

March 11, 2026

Remote collaboration has become increasingly common in software development, yet it often lacks the rich non-verbal cues that support effective communication in collocated settings. In pair programming, where two developers must continuously coordinate their attention and reasoning, the inability to see what a partner is focusing on creates challenges in referencing and understanding code locations. Existing remote tools such as screen sharing or distributed IDEs provide limited support for such coordination, often forcing developers to rely on explicit and sometimes inefficient communication strategies like verbalizing line numbers or function names. This gap highlights the need for mechanisms that can restore shared awareness and improve interaction quality in remote pair programming environments.

To address this limitation, the paper introduces a shared gaze awareness system that visualizes where each programmer is looking within the code. By leveraging eye-tracking technology, the system provides an unobtrusive visual cue that helps collaborators align their attention in real time. The study demonstrates that such visualization improves coordination by increasing the amount of time pairs focus on the same code regions, enabling more efficient and implicit communication. As a result, developers can respond more quickly and accurately to references, reducing the need for overly explicit descriptions. This work suggests that incorporating gaze-based awareness into remote collaboration tools can significantly enhance communication and make remote pair programming more effective and natural.

## 1 Motivation

Remote work has become increasingly prevalent due to its flexibility and ability to support diverse lifestyles, yet it lacks critical elements of face-to-face collaboration such as non-verbal cues and gestures. In collaborative programming tasks, especially pair programming, these cues play a crucial role in establishing shared understanding and coordinating attention. Without the ability to naturally perceive where a partner is looking, remote collaborators face difficulties in referencing specific locations in code and maintaining synchronized workflows. Prior research highlights that eye gaze is tightly coupled with communication, where speakers and listeners use gaze to establish common ground and disambiguate references [1, 2]. The absence of such cues in remote environments motivates the need for mechanisms that can restore this lost dimension of interaction.

Pair programming, a widely adopted collaborative practice, relies heavily on continuous coordination, shared attention, and effective communication between partners. It has been shown to improve code quality, focus, and developer satisfaction [3, 4]. However, its effectiveness is often tied to physical co-presence, making it less common in remote settings. Survey results reported in the paper indicate that while developers value pair programming, very few engage in it remotely due to limitations in current tools. General-purpose tools like video conferencing and screen sharing fail to provide adequate support for non-verbal communication, while specialized tools offer only limited interaction capabilities. This disconnect between the benefits of pair programming and the constraints of remote work environments motivates the exploration of new interaction techniques.

To bridge this gap, prior work in collaborative systems has explored the role of shared gaze awareness as a means of enhancing coordination and communication. Studies have shown that sharing gaze information can improve task performance, increase joint attention, and enable more efficient use of implicit references

such as deictic expressions [5, 6]. These findings suggest that gaze can serve as a powerful referential cue in collaborative tasks. Building on this foundation, the paper is motivated to investigate how shared gaze awareness can be applied specifically to remote pair programming, with the goal of improving communication efficiency, reducing ambiguity, and ultimately making remote collaboration as effective as its collocated counterpart.

## 2 Methodology

**Overview** The study employs a controlled experimental design to evaluate the effectiveness of a shared gaze awareness visualization in remote pair programming. The primary objective is to understand how displaying a collaborator’s gaze influences coordination, communication, and task performance. The experiment follows a within-subjects design, where each pair of participants completes programming tasks under two conditions: with and without gaze awareness. This allows for direct comparison of behaviors across conditions while minimizing variability due to participant differences.

**Study Design** A  $2 \times 2$  counterbalanced within-subjects design was used, incorporating two independent variables: gaze awareness (enabled vs. disabled) and task type. Each participant pair performed two refactoring tasks, experiencing both experimental conditions in different orders to control for learning effects. The tasks were designed to be equivalent in difficulty and duration, ensuring that differences in outcomes could be attributed to the presence of gaze visualization rather than task complexity.

**Participants** The study involved 24 professional software developers who were familiar with programming in C#. Participants were recruited from a large technology company and compensated for their time. They were paired and asked to collaborate in a simulated remote environment. All participants provided consent and had prior experience relevant to the programming tasks used in the experiment.

**Experimental Setup** To simulate remote pair programming, the researchers created a controlled environment using a single computer with dual monitors placed back-to-back, preventing participants from seeing each other while allowing verbal communication. Each participant had their own keyboard, mouse, and display, along with an eye-tracking device.

A custom extension for the development environment synchronized both participants’ views of the code, ensuring that scrolling, highlighting, and navigation actions were mirrored. The system also captured gaze data from both users and displayed a visualization indicating where each partner was looking. This setup allowed the researchers to isolate the effect of gaze awareness while maintaining a realistic collaborative workflow.

**Tasks** Participants first completed a short familiarization task, making minor changes to a codebase to understand its structure. This task was not included in the analysis.

They then performed two refactoring tasks on a C# codebase resembling a simple game. Each task lasted approximately 15 minutes and required collaboration to improve code quality and structure. One task was completed with gaze visualization enabled, and the other without it.

**Gaze Visualization Intervention** The experimental condition introduced a shared gaze visualization that displayed a colored rectangular indicator representing the partner’s focus area in the code. The visualization was designed to be subtle and non-intrusive, highlighting a small vertical region in the code editor. It also changed color when both participants were looking at the same region, indicating shared attention.

The control condition removed this visualization while keeping all other aspects of the setup identical.

**Data Collection** Multiple forms of data were collected during the experiment:

- Eye-tracking logs capturing gaze positions and overlap between participants
- Screen recordings and video recordings of participants’ interactions

- Interaction logs tracking navigation and editing behavior
- Post-task questionnaires measuring participants’ perceptions of collaboration
- Interviews conducted after the tasks to gather qualitative feedback

**Communication Analysis** The recorded sessions were manually analyzed to identify references and acknowledgments during communication. Each reference to a code location was categorized based on how explicitly it was expressed, ranging from implicit (e.g., “this” or “here”) to explicit (e.g., line numbers or specific function names).

Acknowledgments were classified as either successful or unsuccessful depending on whether the partner correctly understood the reference. This analysis enabled the researchers to study how gaze awareness influenced communication style and effectiveness.

**Gaze Analysis** A key metric in the study was gaze overlap, defined as the extent to which both participants were looking at the same region of code simultaneously. Each participant’s focus area was approximated as a small vertical window centered on their gaze position. Overlap was recorded when these regions intersected for a minimum duration to avoid noise from rapid eye movements.

Additionally, gaze overlap was used to infer implicit acknowledgments of references when no verbal confirmation was given, providing insight into non-verbal coordination.

**Evaluation Metrics** The study evaluated the impact of gaze awareness using several quantitative and qualitative measures:

- Fraction of time spent in shared gaze overlap
- Distribution of reference types (implicit vs. explicit)
- Success rate of communication references
- Time taken to acknowledge references
- Participant perceptions from questionnaires and interviews

These metrics collectively assess how gaze awareness affects coordination, communication efficiency, and overall collaboration quality in remote pair programming.

### 3 Results

The results evaluate how shared gaze awareness affects coordination, communication, and efficiency in remote pair programming. The analysis focuses on three primary aspects: the extent of shared attention, the nature of communication between participants, and the effectiveness and speed of reference resolution. The findings consistently indicate that gaze awareness improves coordination and enables more efficient interaction, even though it does not significantly impact task completion time.

**Gaze Overlap and Shared Attention** The introduction of gaze visualization significantly increased the amount of time participants spent looking at the same region of code. Pairs using gaze awareness showed nearly double the proportion of shared gaze compared to when it was absent. This indicates stronger synchronization of attention between collaborators. Increased gaze overlap suggests that participants were better aligned in their understanding of the task and were more effectively coordinating their actions in real time.

**Communication Patterns** The presence of gaze awareness altered how participants referred to locations in the code. Specifically, there was a substantial increase in the use of implicit (deictic) references, such as “this” or “here,” when gaze visualization was enabled. At the same time, the use of more explicit references, such as naming functions or specifying line numbers, decreased. This shift demonstrates that participants relied on gaze as a shared contextual cue, reducing the need for detailed verbal explanations.

The visualization effectively acted as a referential pointer, allowing collaborators to communicate more naturally and with less effort.

**Reference Success Rate** The effectiveness of communication improved with gaze awareness. A higher proportion of references were successfully understood and acknowledged when the visualization was present. Notably, implicit references—typically more ambiguous—became significantly more reliable. This indicates that gaze awareness not only changes how participants communicate but also enhances the clarity of that communication. By providing a shared visual context, the system reduces ambiguity and increases the likelihood that references are interpreted correctly.

**Speed of Acknowledgment** Participants were faster at responding to references when gaze visualization was enabled. On average, references were acknowledged more quickly compared to the condition without gaze awareness. Interestingly, while implicit references did not show a significant improvement in speed individually, more explicit references (such as line numbers or specific names) were acknowledged faster. This suggests that gaze awareness enhances overall responsiveness, even when traditional explicit communication is used.

**Task Completion Time** Despite improvements in coordination and communication, there was no significant difference in task completion time between the two conditions. All participant pairs used the full allotted time for the tasks regardless of whether gaze awareness was available. This outcome is attributed to participants’ tendency to continue refining and improving the code beyond the minimum requirements, reflecting professional standards rather than inefficiencies in collaboration.

**Participant Perceptions** Post-task questionnaire results showed a slight improvement in participants’ perception of their partner’s focus when gaze awareness was enabled. Participants felt that their collaborators were more attentive and aligned with the task. However, most other subjective measures of collaboration quality did not show significant differences. This suggests that while the benefits of gaze awareness are measurable in behavior, they may be subtle and not immediately perceived by users.

**Qualitative Insights** Interviews with participants revealed that gaze visualization was particularly useful for understanding where a partner’s attention was directed. Many participants described it as a substitute for pointing gestures, helping them quickly establish shared context. Some participants also noted that the color change indicating shared focus reinforced their sense of being “on the same page.” Interestingly, several participants reported that they were not consciously aware of using the visualization, even though their behavior reflected its influence. This highlights the subtle and unobtrusive nature of the design.

## 4 Implications

The findings suggest that shared gaze awareness can significantly enhance coordination in remote pair programming by restoring an important non-verbal communication channel that is typically lost outside collocated settings. By enabling developers to see where their partner is looking, the system supports faster grounding of shared context and reduces ambiguity in communication. This allows collaborators to rely more on natural, implicit forms of interaction rather than verbose and explicit descriptions, making remote collaboration feel more fluid and intuitive. As a result, gaze-based cues can act as a lightweight yet powerful mechanism to improve communication efficiency without disrupting existing workflows.

Beyond pair programming, the study highlights broader design implications for collaborative systems. The effectiveness of a subtle and non-intrusive visualization suggests that awareness tools should be carefully integrated to complement, rather than distract from, the primary task. The approach can be extended to other collaborative domains such as document editing, learning environments, and distributed teamwork, where shared attention is critical. Additionally, the results emphasize the importance of designing interaction techniques that align with task structure and user behavior, indicating that gaze awareness systems should be adaptable and context-sensitive to maximize their utility.

## 5 Threats to Validity

One key limitation lies in the accuracy and reliability of eye-tracking technology. Eye trackers are inherently imprecise and can be affected by user movement, calibration issues, and hardware differences. Although the study accounted for this by designing a broader visualization region, inaccuracies in gaze data could still influence the results. Additionally, the use of two different eye-tracking devices introduces potential inconsistencies in measurement, which may affect the comparability of gaze data across participants.

Another threat concerns the experimental setup and generalizability of results. The study simulated remote collaboration using a controlled environment with no communication latency and synchronized displays, which differs from real-world remote settings that often involve network delays and varying system configurations. Furthermore, the experiment focused on a mirrored-view setup rather than fully distributed workflows where collaborators may work on different parts of the codebase. These factors limit the external validity of the findings and suggest that further studies are needed to evaluate the approach in more realistic and diverse environments.

## References

- [1] Zenzi M. Griffin and Kathryn Bock. “What the eyes say about speaking”. In: *Psychological Science* 11.4 (2000), pp. 274–279.
- [2] Joy E. Hanna and Susan E. Brennan. “Speakers’ eye gaze disambiguates referring expressions early during face-to-face conversation”. In: *Journal of Memory and Language* 57.4 (2007), pp. 596–615.
- [3] Jo E. Hannay et al. “The effectiveness of pair programming: A meta-analysis”. In: *Information and Software Technology* 51.7 (2009), pp. 1110–1122.
- [4] Andrew Begel and Nachiappan Nagappan. “Pair Programming: What’s in It for Me?” In: *ESEM*. 2008, pp. 120–128.
- [5] Susan E. Brennan et al. “Coordinating cognition: The costs and benefits of shared gaze during collaborative search”. In: *Cognition* 106.3 (2008), pp. 1465–1477.
- [6] Bertrand Schneider and Roy Pea. “Real-time mutual gaze perception enhances collaborative learning and collaboration quality”. In: *International Journal of Computer-Supported Collaborative Learning* 8.4 (2013), pp. 375–397.